

DSA Track: Joint Process Write-Up (Top-K Indexer + Sparse Attention)

Author: Yury Kirpichev / Team Wombat Tracks: - dsa_topk_indexer_fp8_h64_d128_topk2048_ps64 (final solution dsa_topk_indexer_fp8_b200_v3, commit 31b8f71) - dsa_sparse_attention_h16_ckv512_kpe64_topk2048_ps64 (final solution wombat_dsa_sparse_mla_sm100a_final, tag submission-final-v2, commit de475b9)

Hardware (both): NVIDIA B200 / sm_100a. Methodology (both): experiment-driven, agent-assisted (Cursor + Claude/GPT-family LLMs); all design decisions accepted on measured B200 numbers.

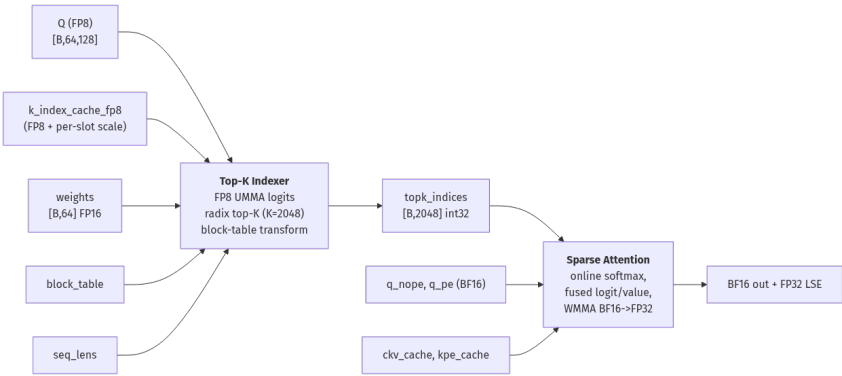
Headline Summary

Field	Top-K Indexer (FP8)	Sparse Attention (BF16)
Datatype	FP8 E4M3 x FP8 E4M3 -> FP32, FP16 logits	BF16 in/out, FP32 LSE and split-K partials
Workloads	128 / 128 PASSED	23 / 23 PASSED, abs_err <= 1.56e-2
vs naive / PyTorch ref	964x mean (unconfirmed; see caveat)	126.03x (unconfirmed; see caveat)
vs FlashInfer/DG	38.4x mean (8.3x worst, 72.3x best) (unconfirmed)	14.18x (unconfirmed)
Aggregate latency	mean 7.83 us, p50 2.40 us, p95 17.80 us (unconfirmed)	0.719 ms total (vs 6.793 ms FlashInfer, 66.079 ms ref)
Source files	<code>solution/python/{solution.py, kernel.cu, tcgen05_ptx.h, umma_desc.h}</code>	<code>solution/python/{solution.py, kernel.cu}</code>

Caveat — comparison numbers are unconfirmed for both kernels. Headline ratios come from runs whose timing backend was not independently verified to be `cupti-python` on each side; sub-10-us kernels are sensitive to event-timing fallback. Both ratios will be remeasured with `cupti-python` forced and the per-kernel timing backend confirmed in the log. The relative ordering and one-Modal-invocation comparisons are believed correct.

DSA Pipeline and Joint Problem Setting

The two operators form one half of a DSA decoding step on B200: the **Top-K Indexer** scores every paged KV position against the current query and emits the K=2048 most relevant token indices; the **Sparse Attention** kernel then reads only those 2048 positions out of a compressed MLA KV cache and produces BF16 output + FP32 LSE in destination-passing style.



The 128 (indexer) and 23 (attention) contest workloads each span small to mid-range shapes where launch overhead, occupancy, memory scheduling, and sparse-index density are at least as important as raw arithmetic throughput. A single-kernel design cannot serve both small and large regimes well — both submissions ended up as **size/density-aware host dispatchers** over a small set of specialized kernels, with a CUDA-graph cache wrapping the launch sequence.

Shared Development Method

Each optimization was introduced on a branch or phase, benchmarked on Modal B200 with `cupti-python` matching the official harness, and either kept, refined, or reverted on measured numbers. This produced a history with substantial **negative results** that constrained the final designs as much as the positive ones: head tiling, deeper value/K prefetch, full warp specialization, BF16 logit storage, TMA gather, and large-N Blackwell UMMA were all tried in the attention kernel and reverted; in-Stage-1 ordered-u16 conversion, fused block-table transform in the emit path, kKVStages 3->4, and a TMEM double-buffer + MMA-overlap scheme were tried in the indexer and reverted. Documenting why these were rejected was an explicit goal of both write-ups.

Development was **agent-assisted**: Cursor with Claude/GPT-family LLMs was used as a pair-programming and refactoring tool; all benchmarks, correctness checks, and commit decisions were owned by the human author and accepted only when measured B200 numbers improved. The agent never had access to contest workload data or evaluator outputs beyond what appeared in development logs.

Kernel A — Top-K Indexer (FP8, H=64, D=128, K=2048)

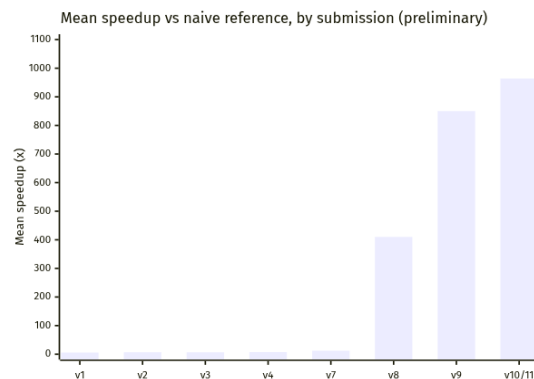
Problem. For each batch row b , compute FP8 dot products $S[h] = \sum_d Q[b,h,d] * K[page,slot,d]$ over 64 heads / 128 dims with FP32 accumulation, weighted ReLU logits $logit[b,t] = scale[page,slot] * \sum_h ReLU(S[h]) * weights[b,h]$ for every position in $[0, seq_lens[b])$, top-K = 2048 selection, and a `block_table` transform back to physical page slots. The KV cache is FP8 with per-slot FP32 scales (132 bytes/slot, 64 slots/page); outputs are pre-allocated (DPS).

The evaluator pre/post-#354 was the dominant constraint

The single largest constraint on the timeline was the evaluator. Before [flashinfer-bench PR #354](#) (merged Apr 10, 2026) there was no dedicated `DsaTopKIndexerEvaluator`. The default evaluator compared **raw output index vectors** element-wise, with two consequences: (1) when multiple positions tie on logit value any custom top-K — radix select, CUB sort, atomic emit — produced a valid top-K set that the evaluator rejected as `INCORRECT_NUMERICAL` because the index ordering did not match the reference's tie-breaking; and (2) the evaluator's random `k_index_cache_fp8` inputs were not packed in the deepgemm FP8 format the FlashInfer baseline expected, causing false negatives against FlashInfer itself. In practice the only top-K path that reliably passed was `torch::topk`.

Pre-#354 (v1-v4, Mar 29 - Apr 6, ceiling ~7x). The early kernel was constrained to ATen ops that preserved bit-exact index agreement: a fused FP8 page-gather + dequant CUDA kernel, then `torch::bmm` for Q·K, ATen `relu` + weighted sum, and `torch::topk`. Several aggressive ideas were tried and reverted because they broke index-level agreement even when the selected values were correct: fused Q·K dot products, per-row scoring, BF16 reduction, chunked GEMM with running top-K.

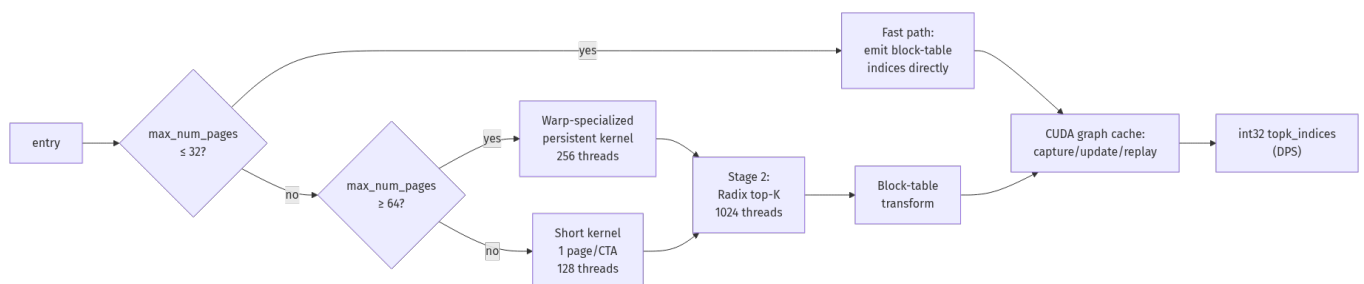
Post-#354 (Apr 11 onward). PR #354 introduced `DsaTopKIndexerEvaluator` which compares **sorted value vectors**, vectorizes index validation (duplicates, out-of-range, block-table reachability), and provides a `build_baseline` with correct FP8 packing. Within ten days the solution was rewritten from scratch: pure PyTorch FP8 dequant + `bmm` indexer (Apr 11-12), then custom CUDA top-K with CUB radix sort and FP8 MMA logits via `mma.sync.m16n8k32.e4m3` PTX (Apr 19-20), then a full SM100a UMMA rewrite with `tcgen05.mma.cta_group::1`, radix-select top-K, and size-aware dispatch (v7, v8, ~16 us mean), then persistent CTAs, warp specialization, Stage-2 SMEM cache, dispatch retuning and a CUDA graph cache (v9-v11). The final 7.83 us mean / 38.4x vs FlashInfer was reached on Apr 24.



The flat plateau at v1-v4 corresponds to the period when the old evaluator capped progress; the sharp rise at v7-v8 follows PR #354 and the full custom-CUDA + UMMA + radix-select rewrite; the final ~13% from v9 to v11 comes from CUDA-graph host-overhead elimination on the small-workload bucket.

Final indexer design

A host dispatcher selects between three Stage-1 variants plus a fast-path bypass; Stage-2 is a radix top-K; a hand-rolled CUDA graph cache wraps the whole launch sequence.



Fast path (`max_num_pages <= 32`). When the entire paged context fits within K=2048 every position is in the top-K, so the output is just a block-table-transformed `[0, seq_len]` padded with -1, independent of Q, K, weights. 128 threads with vectorized `int4` stores, ~2.3 us including graph replay overhead.

Short kernel (`33 <= max_num_pages < 64`). One CTA per page, 128 threads. Q and K loaded into 128B-swizzled SMEM, TMEM allocated, UMMA issued with 4 K-iterations on $(M=128, N=64, K=32)$, FP32 accumulator read back, ReLU-weighted logits computed, FP16 emitted.

Warp-specialized persistent kernel (`max_num_pages >= 64`). 256 threads = producer warpgroup (warps 4-7, 40 regs, 3-stage `cp.async K` pipeline) and math warpgroup (warps 0-3, 232 regs, issues UMMA, reads TMEM, computes ReLU/weighted sum, emits). Q is loaded once and TMEM allocated once per CTA. Per-stage

`K_ready[buf]/K_done[buf]` mbarriers drive the handoff. A subtle bring-up bug: `cp.async.wait_group` is per-thread, so a producer-warpgroup `bar.sync` is required before the single-thread `mbarrier.arrive` on `K_ready[buf]` to ensure all threads' `cp.async` writes are visible.

Stage-2 radix top-K. Two-round radix select over 16-bit ordered keys finds the pivot in $O(\text{seq_len})$. Since $\text{seq_len} \leq 16384$ for all benchmarked workloads, ordered keys are cached in a 32 KB SMEM array on round 0 and re-read on subsequent passes (4x fewer HBM reads). Elements $>$ pivot emit first, equal-to-pivot fills remaining slots, then the block-table transform runs.

Critical-path finding (negative result)

The most useful single finding was that the per-tile critical path in the persistent kernel is **dominated by the TMEM-to-register transfer** (`tcgen05_ld_32x32b_x64_b32 + tcgen05_wait_ld`), not by MMA compute, producer-prefetch latency, or the FMA accumulation chain. Levers that were measured and rejected: TMEM double-buffer + MMA-issue overlap (+3.6-5.2%, MMA not on critical path); kKVStages 3 $\rightarrow$ 4 (flat); 4-way accumulator split (within noise); block-scaled UMMA `mx8f6f4` (infeasible — per-row FP32 scales, not per-K-block UE8M0); host-side `seq_lens.max()` sync (+20 us). Every micro-optimization that added work to the post-`wait_ld` emit path regressed the 40-63 page bucket by 0.6-3.7% by lengthening the critical path between successive TMEM loads. The only structural way to reduce `wait_ld` cost is to shrink `kUMMA_N` via 2-CTA cluster UMMA, deferred due to infrastructure complexity (PTX wrappers, cluster launch, DSMEM, cluster-scoped mbarriers). What did stick in Phase 3: Stage-2 ordered-u16 SMEM cache, int4-vectorized fast-path stores, and re-tuning the persistent dispatch threshold from ≥ 40 to ≥ 64 (recovers 8.9% on the 40-63 page bucket where the short kernel saturates B200's 148 SMs faster than the persistent pipeline).

Kernel B — Sparse Attention (BF16, H=16, ckv=512, kpe=64, K=2048)

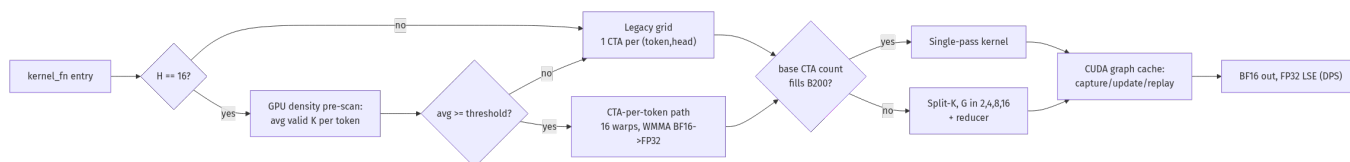
Problem. The operator receives per-token query tensors `q_nope` and `q_pe`, compressed KV caches `ckv_cache` and `kpe_cache`, and `TOPK=2048` sparse indices. It writes a BF16 output tensor and FP32 log-sum-exp tensor in destination-passing style. The practical challenge was not arithmetic intensity but the workload distribution: many shapes are small enough that launch overhead, occupancy, and sparse-index density dominate raw throughput.

Optimization story (eight phases)



Three principles survived to the final solution: **(1) Reduce repeated HBM traffic** — `v5_fused` collapsed logit, softmax, and value accumulation into a single-pass online-softmax K loop that consumes each sparse KV tile exactly once. **(2) BF16 only at the output boundary** — intermediate logits / split-K partials in BF16 caused intermittent `abs_err` spikes; the final split-K path uses FP32 partial buffers and converts to BF16 only on the final write. **(3) Treat workloads as shape/density families, not one regime** — three correct paths plus a GPU-side density gate.

Final attention design



Legacy kernel. One CTA per (token, head). Sparse indices are prescanned with vectorized int4 loads; invalid entries are canonicalized to -1 and zero-filled in shared memory rather than fetched. The K loop is fused (load tile $\rightarrow$ logits $\rightarrow$ online softmax update $\rightarrow$ value accumulate), keeping memory traffic close to one use per tile and never materializing a full TOPK logit array.

Split-K. For grids that underfill B200, a split factor `G` in $\{2,4,8,16\}$ runs `G` partial kernels emitting FP32 partial output, max, and normalizer buffers; a reducer combines the `G` online-softmax states with the same max/sum rescaling identity and writes BF16 + FP32 LSE.

CTA-per-token (H=16). One CTA per token, one warp per head, with WMMA BF16xBF16 to FP32 on both logit and value-accumulation steps. This converts the 16-head batch into tensor-core-shaped work while keeping online-softmax state per head. MMA scratch and output scratch share SMEM overlays because their lifetimes are disjoint. This was the main late-stage win over the scalar per-head path.

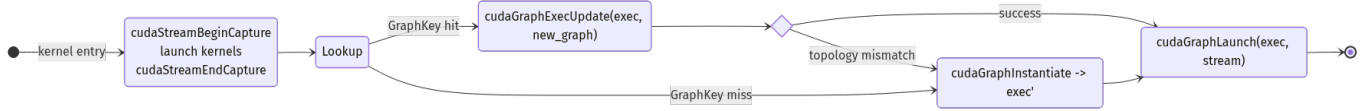
Density pre-scan. A GPU pre-scan counts valid sparse-index entries and routes high-density shapes to the CTA-per-token path; low-density shapes (e.g. 385742b2, 4c46a94b) fall back to the legacy grid, avoiding the fixed CTA-per-token floor.

Negative results

Blackwell `tcgen05.mma/UMMA` expected far larger `N` than the skinny-GEMM-shaped value accumulation here provides; full warp specialization was correct but issue-slot limited; TMA/gather variants and deeper pipelines lost to the simpler `cp.async` path. The final design is deliberately not "use every Blackwell feature." This is the **opposite** finding to the indexer, where UMMA was the right tool — the contrast is structural: the indexer's Stage-1 has a square-ish (`M=128`, `N=64`, `K=32`) shape that maps cleanly to UMMA, while the attention's value accumulation is a skinny GEMM along a small head dimension where UMMA's overhead dominates.

Cross-cutting: shared CUDA graph cache pattern

Both kernels use the same hand-rolled in-extension graph cache. `do_bench` clones inputs every iteration so pointers change but shapes (and dispatch plan) stay fixed; for small workloads where the kernel itself is single-digit microseconds, ~ 2.5 us per `cudaLaunchKernel` dominated. The cache is keyed by (`stream`, `dispatch_path`, `shape`, `scale_bits`, `split_factor`, ...) (no tensor pointers). Each call captures a fresh graph with current pointers and tries `cudaGraphExecUpdate` against the cached exec; on topology mismatch the entry is re-instantiated. Falls back to direct launch if `cudaStreamBeginCapture` fails. In the indexer this collapsed the fast-path bucket from 6.5 us to 2.3 us mean (caveat: the disambiguation between graph-cache effect and timing-methodology change is incomplete — see validation below).



Validation and Measurement Limitations

All 128 indexer workloads pass the post-#354 `DsaTopkIndexerEvaluator`; all 23 attention workloads pass with `abs_err <= 1.56e-2`. Final benchmarks for both were collected on Modal B200 with the comparison harness configured for `cupty-python` (warmup 3, iterations 100, 5 trials).

Both kernels' headline ratios are flagged as unconfirmed. It has not been independently verified that `cupty-python` was active on every side of every comparison rather than the harness silently falling back to CUDA-event timing; event overhead is significant for sub-10-us kernels, so any side that fell back would inflate that side's ratio. Both ratios will be remeasured by rerunning with `cupty-python` forced and confirming the per-kernel timing backend in the log.

For the attention kernel specifically, the **isolated contribution of the in-kernel CUDA graph cache was not disambiguated**. The `vs_prev` column in the comparison report contrasts current `cupty-timed` latencies against an earlier `WOMBAT_CUDA_GRAPH=0` run that was CUDA-event timed, conflating timing methodology with cache on/off. The disable knob was removed before the cleanup window closed, so a clean `cupty-timed` A/B was not produced; this report does not claim a specific speedup for the attention graph cache in isolation.

Post-deadline disclosure: Sparse Attention density-count cache

After the deadline, a closer look at the sparse-attention host-side density pre-scan disclosed a design point worth flagging. The pre-scan launches a small auxiliary kernel that counts valid sparse-index entries, copies the count back via `cudaMemcpyAsync(D->H) + cudaStreamSynchronize`, and uses it to choose between the legacy and CTA-per-token paths. To amortize the host round-trip, a small host-side cache (`DensityCache`, `N=32`) keys the count by (`sparse_indices.data_ptr()`, `T`, `num_kv_rows`) and reuses prior counts across calls. Read strictly against the FAQ rule "buffer contents are recalculated on every run, not caching results from a previous call", this is **borderline** — the cached value is dispatch metadata only (both paths produce numerically equivalent output, all 23 workloads pass either way), but it does reuse a previous auxiliary-kernel result. **Any rule violation here was unintentional.**

A six-configuration B200 A/B (`artifacts/density_cache_vs_shape_dispatch/REPORT.md`) re-measured the cache against a sync-free shape-only dispatch (`T >= 8 => CTA-per-token`, else legacy). Mean speedup across the 23 workloads, normalized to a common reference: **cache ON 156x** (current submission, sync-free in steady state because the cache hits), **cache OFF 51x** (recompute every call, full sync), **shape rule 157x** (sync-free, FAQ-clean, slightly beats the cache by avoiding two `T=6` mis-dispatches the `density_avg >= 210` threshold introduced on `ddfa9e34` and `d57eb9e1`). The recommended cleanup for any future revision is to delete the `DensityCache` infrastructure (~ 150 LoC) and replace it with a one-line shape check `if (H == CPT_H && T >= 8) path = CPT;` — same speed, no `cudaStreamSynchronize`, no reliance on PyTorch caching-allocator pointer reuse, unambiguously FAQ-compliant.

In summary, the joint submission combines, on both kernels, memory-traffic reduction, fused critical loops (online softmax in attention; UMMA + radix-select pivot in the indexer), shape/density-aware host dispatch, and CUDA graph replay for launch overhead. Both are self-contained (`solution.py` + `kernel.cu` (+ PTX/UMMA headers in the indexer)), JIT-compiled via `torch.utils.cpp_extension.load` with `-gencode arch=compute_100a,code=sm_100a`. Both pass all public workloads.

References

Submitted source (Top-K Indexer) — pinned to submission-v11 (commit 31b8f71) of ykirpichev/dsa_topk_indexer_fp8_h64_d128_topk2048_ps64

1. Final kernel implementation - [solution/python/kernel.cu](#).
2. Python entry point and JIT compile flags - [solution/python/solution.py](#).
3. Inline PTX wrappers for SM100a - [solution/python/tcgen05_ptx.h](#).
4. UMMA descriptor layouts (vendored from CUTLASS) - [solution/python/umma_desc.h](#).
5. Submission manifest - [config.toml](#).

Submitted source (Sparse Attention) — pinned to submission-final-v2 (commit de475b9) of ykirpichev/dsa_sparse_attention_h16_ckv512_kpe64_topk2048_ps64

1. Final kernel implementation - [solution/python/kernel.cu](#).
2. Python entry point and JIT compile flags - [solution/python/solution.py](#).
3. Submission manifest - [config.toml](#).

Contest documentation

1. Top-K indexer track docs - [README.md](#), [FAQ.md](#), [EVALUATION.md](#).
2. Sparse attention track docs - [README.md](#), [FAQ.md](#), [EVALUATION.md](#), [SUBMISSION.md](#).

Performance and correctness artifacts

1. Top-K indexer final cupti-timed comparison report - [reports/submission-v10.md](#), raw log [reports/submission-v10-raw-cupti.log](#).
2. Top-K indexer optimization history and ablations - [IMPROVEMENTS.md](#), [experiments/FUTURE_IDEAS.md](#).
3. Top-K indexer milestone tags - [submission-v1](#) ... [submission-v10](#), [submission-v11](#) (full chronological history: [git log](#)).
4. Sparse attention milestone tags - [submission-v1](#), [submission-final](#), [submission-final-v2](#).
5. Sparse attention artifacts (cupti-timed comparison report, raw Modal log, graph-cache A/B logs, ablations) live under [artifacts/](#) in a follow-up branch tied to [submission-final-v2](#).
6. Sparse attention post-deadline density-cache vs sync-free shape-dispatch A/B - [artifacts/density_cache_vs_shape_dispatch/REPORT.md](#) (six-configuration B200 sweep, per-workload latency table, cache hit/miss trace, recommended cleanup).

External references

1. Flashinfer-bench PR #354 - DsaTopkIndexerEvaluator for correct tie-breaking comparison: [github.com/flashinfer-ai/flashinfer-bench/pull/354](#).
2. FlashAttention-4 (Dao et al., arXiv 2603.05451, Mar 2026) - asymmetric scaling, 2-CTA MMA, TMEM intermediates, warp specialization.
3. Dao et al., "FlashAttention", NeurIPS 2022 - [arXiv:2205.14135](#). FlashAttention-2 - [arXiv:2307.08691](#).
4. Hong et al. / FlashDecoding, "Flash-Decoding for long-context inference" - [crfm.stanford.edu/2023/10/12/flashdecoding.html](#).
5. NVIDIA CUTLASS Blackwell docs - `tcgen05.mma.kind::f8f6f4`, `cta_group=2`, `SmemDescriptor/InstrDescriptor` layouts.
6. Colfax Research - "Writing GEMM Kernels Using Tensor Memory For Blackwell GPUs."
7. DeepGEMM `sm100_fp8_paged_mqa_logits.cuh` - reference production kernel with warp specialization, TMA pipelines, persistent scheduling.
8. NVIDIA CUDA C++ Programming Guide - CUDA graphs / `cudaGraphExecUpdate` ([link](#)) and `nvcuda::wmma` ([link](#)).
9. PyTorch JIT C++/CUDA extensions, `torch.utils.cpp_extension.load` - [pytorch.org/docs/stable/cpp_extension.html](#).
10. FlashInfer attention library (baseline used by both contest harnesses) - [github.com/flashinfer-ai/flashinfer](#).
11. MLSys 2026 FlashInfer AI Kernel Generation Contest entry definitions - [github.com/flashinfer-ai/mlsys26-contest](#).

Tools and Languages

Tool / Language	Role
CUDA C++ (sm_100a)	All kernels: UMMA / WMMA via inline PTX, cp.async pipelines, radix top-K, online softmax
<code>torch.utils.cpp_extension.load()</code>	JIT compilation with <code>-gencode arch=compute_100a,code=sm_100a</code>
Inline PTX (indexer <code>tcgen05_ptx.h</code>)	TMEM alloc/dealloc, UMMA issue/commit/wait, cp.async, mbarrier, warpgroup reg reconfig
CUTLASS bitfield layouts (<code>umma_desc.h</code>)	SmemDescriptor and InstrDescriptor for UMMA, vendored for self-containment
Python (<code>solution.py</code>)	Thin wrapper: JIT-compiles and calls the extension
Modal	Cloud B200 access for benchmarking
<code>cupti-python</code>	GPU-side timing (matches official evaluation methodology)
Cursor + LLM agents	Agent-assisted development (see disclosure below)

AI / Agent-Assisted Development Disclosure

Both submissions were agent-assisted (Cursor + Claude/GPT-family LLMs); direction and acceptance criteria were human, all changes were accepted on measured B200 numbers. The agent was used as a pair-programming and refactoring tool for code generation, inline PTX wrapper authoring, and experiment scaffolding. All design decisions (kernel architecture, dispatch thresholds, pipeline depth, optimization accept/reject) were made by the human author based on benchmark measurements. The agent never had access to contest workload data or evaluation results beyond what was visible in the development logs. Full sparse-attention statement: [REPORT_AI_DISCLOSURE.md](#).